

Use Case Review Standard Operating Procedure

Melbourne Open Playground (MOP)

1. Purpose and Scope

This Use Case Review Standard Operating Procedure (UCRSOP) standardises the process a Quality Manager takes when reviewing a usecase. The goal of a quality review is to assess the usecase against the [Usecase Quality Tracker](#), provide clear and actionable tasks when an update is needed and create an MS Planner Card for this update.

For the definition and purpose of a usecase, refer to the section “**What do we mean when we say ‘usecase’**” in the [Melbourne Open Playground \(MOP\) Project Guide](#). In summary, a usecase is an interactive .ipynb notebook tutorial that uses open data to address real-world problems, adding value and explaining how the data is analysed at each step so others can apply similar approaches.

This UCRSOP steps through the process of reviewing a usecase from beginning to end, from selecting a usecase, undertaking the review, providing the detailed outcome of the review, and creating the MS Planner card with the steps needed to update the usecase. It is important to follow these steps so that all quality reviews are consistent and evidence based.

2. What the ‘QA Checklist V2’ Covers

The checklist (sheet ‘QA Checklist V2’) is one row per usecase. Complete each column as you move across. The table below explains in more detail what you should be looking for in each column.

QA Checklist V2 headers	Explanations	Example
usecase code	Unique identifier for the notebook. Do not change it.	UC00001
usecase name	Record the notebook’s published name and check whether it matches the title within the use case. Make sure the usecase code is at the start of the file name.	UC00010_2023_Melbourne_Liveability_Assessment
Domain (Dropdown)	Select the domain that best represents the main problem, intended outcome, and datasets of the usecase. If a usecase fits into multiple domains, select the domain most directly addressed by its scenario and analysis. Refer to <u>Domain Selection Guide</u> for definitions and examples.	Transport and Mobility
Comments	Leave unchanged – Mostly just additional information about the use case.	NEEDS RE-REVIEWING
Reviewer (Dropdown)	Select your own name before starting the review.	{Quality Reviewer Name}
Date Reviewed (Dropdown)	Select the current trimester and year.	T2 2026
Review Status (Dropdown)	Default to “ Not Reviewed ” - Select “ Review in Progress ” when starting; - Select “ Review Complete ” only after the checklist and Planner card are complete; - Select “ Update in Progress ” when a Data Scientist is working on the required fixes and create a pull request;	Review in Progress

QA Checklist V2 headers	Explanations	Example
	- Select "Update Complete" when all required updates have been completed and merge to GitHub repository; - Select "Retired" when the usecase cannot be fixed or no longer suitable for publication;	
Overall Rating (Dropdown)	This is something to come back to complete at the end of the review. Give an overall rating out of 5 based on review results. Provide a rating based on the number of quality tracker items marked Yes. "0.5": 6 or fewer Yes responses. "1.0–2.0": More than 6 and up to 12 Yes "2.0–3.0": More than 12 and up to 18 Yes "3.0–4.0": More than 18 and up to 24 Yes "4.0–5.0": More than 24 Yes.	3.5
Planner Card (URL)	Add the MS Planner Card hyperlink after creating it. Display only the usecase code as the linked text. Refer to Step 4 for more details on how to create an MS Planner Card.	<u>UC0000XXX</u>
Does the usecase file name match the title within the notebook and dependency folder?	Check that the same usecase name is used consistently in all relevant files and file names. Provide the locations that it does not.	No - the notebook title says "2023_Melbourne_Liveability_Assessment", but the usecase title within the notebook is "_City Living Comfort".
Does the name adequately reflect what the usecase is about? And is there a year/date range?	The name should clearly describe the topic that the usecase covers and include a relevant date or year range where applicable. Each name should: - Have an assigned unique prefix (e.g. UC00001). - Using underscores "_" instead of spaces or "-". - Be less than 70 characters.	No - "City_Living_Comfort" is too broad, has no date range and does not make sense. It should be "UC00010_2023_Melbourne_Liveability_Analysis"

QA Checklist V2 headers	Explanations	Example
Are all the key areas included at the top: - scenario - what this usecase will teach you - background - data sets - contents	Verify that the notebook begins with the required usecase title and metadata, followed by the five required unnumbered sections in this exact order, also check that there are not additional unnecessary fields, strictly follow the <u>MOP usecase-TEMPLATE</u>. Scenario must use a clear user-story format to explain the real-world problem, intended user, and purpose of the analysis. What this usecase will teach you must describe the learning outcomes and skills the reader will gain. Background must provide concise context or rationale that helps a non-technical reader understand the problem and approach. Data Sets must list every dataset used, include a working source or dependency-folder link, and may include a brief description of each dataset. Contents must appear after Data Sets and before the analysis begins; It must match the actual numbered analysis headings and include clickable links to the corresponding sections.	Partly - Background and Contents are missing. There is an additional "Key Outcomes" section that needs to be removed.
Is the "Time to Complete" appropriate?	Check that the stated completion time is realistic for the intended audience to read the explanations, run the code, inspect outputs, and complete the analysis. Consider the notebook's length, data-download or API time, data preparation, modelling complexity, visualisations, and any prerequisite setup. As a guide: • Short tutorials: 15–30 minutes. • Multi-dataset analysis and visualisation: 30–60 minutes. • Modelling or spatial analysis: 60–120 minutes. • Large-data or multi-modelling: above 2 hours.	Partly - 15 minutes is unrealistic for a notebook of this length. 120 minutes is more appropriate for a notebook with a regression model and evaluation.
Is the difficulty "Level" appropriate?	Check whether the stated level matches the assumed prior knowledge, coding complexity, data preparation, analytical methods, and tools used. A Beginner usecase should provide guided foundational steps. An Intermediate usecase may assume basic Python and data-analysis knowledge. An Advanced usecase may use complex modelling, geospatial methods, or specialised techniques.	No - labelled Beginner, but requires API calls, clustering, and geospatial analysis. Advanced is more appropriate.

QA Checklist V2 headers	Explanations	Example
	If the level is unsuitable, recommend the most appropriate level.	
Are the "Pre-requisite Skills" appropriate?	Check that the notebook lists only the knowledge and tools readers need before starting the usecase. For example: • Programming language knowledge: basic Python and Jupyter Notebook; • Data handling: loading, cleaning, filtering, and merging data. • Visualisation: Matplotlib or Seaborn. • Statistics: t-Test, Skewness or Ranking Test. • Modelling: KNN, CNN or LSTM • Geospatial analysis: GeoPandas. Do not list skills that the notebook itself will teach. If skills are missing or unnecessary, specify which skills should be included or removed.	Partly - "Python" is listed, but basic visualisation, classification modelling knowledge are also needed, these should be added.
Does the "Scenario" clearly outline the purpose / goal of the usecase? If not, please specify.	The Scenario gives a clear statement about: the real-world problem, core challenge and why it matters, proposed analytical solution, expected outcome, its values and who will benefit from the findings. It must align with the downstream data, methods, and conclusion.	Yes. It clearly explains the problem that the usecase is attempting to solve
Is the target audience of the usecase defined in the scenario? If not, please specify.	Check that the Scenario identifies the intended reader, user, or beneficiary of the analysis, such as local business owner, researcher, student, or community staff. The audience should be specific enough to show who will use the findings and why the usecase is useful to them.	Yes - It states the usecase is aimed at Urban Planners.
Are the datasets and their links provided? And are they at the top of the notebook in the appropriate place as per the previous check?	• Verify that all datasets have the correct name, are linked, and listed before the contents section. • Click each hyperlink to confirm that it works and directs users to the current dataset information page, catalogue page, or documentation—not only an API endpoint. • Confirm that the listed datasets match those used in the code.	No - the pedestrian-count dataset is used in the usecase but is not listed in the Data Sets section. The second dataset is linked but the link takes you to the API for the dataset, not the information page.
Is there a "Conclusion" in the notebook? The conclusion should cover achievements, what was learnt, and potential future opportunities.	Ensure that the Conclusion section is properly provided at the end of notebook and includes: - What we achieved in this analysis - What we learned from this analysis - Observations for further opportunities Each part should summarise the actual analysis and results, rather than repeat generic statements.	Yes - all three sections are addressed.

QA Checklist V2 headers	Explanations	Example
Does the conclusion address how the analysis solves the problem stated in the scenario?	Check that the Conclusion explicitly connects the findings back to the problem in the Scenario. It should explain what answer, evidence, insight, or decision support the analysis provides for the intended audience.	Yes - the conclusion clearly outlines how the analysis has addressed the problem stated in the scenario.
What API method is being used? And is this an appropriate way to call the data?	Identify the data-access method and assess whether it is suitable, current, and reliable. The preferred method is always to use an API. Do not accept processes that download API data to local storage before reading it. Search the notebook for local file-reading functions, such as <code>pd.read_csv()</code> , <code>pd.read_excel()</code> , <code>pd.read_json()</code> , or <code>open()</code> , to check.	Yes - Uses Melbourne Open Data API v2 with requests appropriately.
If no API used. Is the dataset being called from a dependency folder with the same name as the usecase?	If the notebook does not use an API, verify that all local datasets are stored in the <u>DEPENDENCIES</u> folder. Check that the dependency folder name matches the usecase Code and Name; verify that the notebook load files using a relative path, for example: <u>DEPENDENCIES/UC00145_Optimising_School_Infrastructure_Planning/dv378_DataVic-SchoolLocations-2024.csv</u> The notebook must not use any personal or hardcoded file paths, such as <u>C:\Users\Name\Desktop\data.csv</u>	N/A - the notebook retrieves data through an API.
Are visualisations clear, relevant, well-labelled, include legends and use the template colours? If not, please specify.	Check that every visualisation supports a specific analytical point. Confirm that the titles state what is being shown; axes, units, categories, and time periods are labelled; legends are included where colours, symbols, or line types need explanation; and labels remain readable. Check that colours follow the <u>MOP usecase-TEMPLATE</u> (colours in this order: Primary Green (#059669), Distinct Purple (#9333EA), Distinct Orange (#F97316), Distinct Blue (#2563EB), Distinct Yellow (#EAB308), and Distinct Pink (#EC4899)).	No - the “Bus Stops by Asset Type” chart has no title, and the template colours have not been used.
Do all the visualisations work? If not, please specify.	Run the relevant cells or inspect their saved outputs. Confirm that each visualisation renders correctly, contains data, and is visible in the published notebook. Check for blank figures, missing legends or tiles, charts that only work after manual interaction.	No - the map in “Data Exploration” section returns a KeyError and does not display.
Are appropriate evaluation metrics used for all analysis? If not, please specify.	Where the notebook uses a measurable analytical method, check that it uses suitable metrics and explains what they mean. Common examples include: Classification: accuracy, precision, recall, F1-score, confusion matrix, and ROC-AUC. Regression: MAE, MSE, RMSE, and R-squared.	Partly - accuracy is reported, but precision and recall are needed for a multi-class classification task.

QA Checklist V2 headers	Explanations	Example
	Clustering: silhouette score, Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI). Time-series forecasting: MAE, RMSE, MAPE, and forecast-versus-actual charts. Ranking or recommendation: precision@k and recall@k.	
Are the libraries & packages used appropriate & minimal?	Review all import statements. Confirm that each library is necessary for the notebook's analysis, visualisations, data access, or output. Import all required libraries once at the beginning of the notebook where practical. Identify duplicate imports, unused libraries, deprecated packages or methods, and imports that can be replaced by an existing library already used in the notebook.	No - io is imported but never used; NumPy is imported twice.
Is there sufficient description between the code blocks and under headings explaining what the code is doing and why it helps to solve the problem stated in the scenario?	Check that each section and meaningful code block has concise explanatory text. The explanation should state what the step does, why it is needed, and how it contributes to conclusions.	No – The sections only provide facts and clinical details about the data; there is no linking back to the problem statement or why the analysis is taking place.
Were there any errors in the code output? If there are errors, please specify.	Run the notebook sequentially from top to bottom. Check for visible errors, critical warnings, undefined variables, missing files, failed API calls, empty responses, incomplete outputs, and out-of-order cell dependencies. Confirm that all charts, tables, and results render properly. If large maps or datasets cause local performance bottlenecks, it is recommended to run the code to test the notebook. This can be done in Google Colab or with GPU instead.	No - Replace print(f'Request failed...') with structured error handling.
Is the code modular, well-organised & commented where needed? If not, please specify.	Identify specific code sections that should be combined, simplified, moved into a function, renamed, or commented. Repeated or complex logic should be placed in clearly named reusable functions where appropriate. Functions should have meaningful names, parameters, return values, and short docstrings explaining their purpose.	Partly - repeated API code should be moved into a reusable function.
Are all output cells clean, with no unnecessary or debug print statements? If not, please specify.	Remove unnecessary outputs, including raw dataset previews, exploratory tables, debug messages, and excessively long print() results. Keep only outputs that support the learning steps or findings.	No - remove the print(df) printout after data cleaning.
Does the code include error handling and edge case checks? If not, please specify.	Check for safe handling of missing data, empty results, failed downloads, division by zero and anything else.	No - check for the empty API response and filter out rows with missing (NA) values before running the analysis.

QA Checklist V2 headers	Explanations	Example
Is the content logically ordered for a beginner, intermediate, or expert?	The explanation, code, and outputs should follow a clear learning sequence. Check that each section builds on the previous one: introduce the purpose and data first, then prepare and analyse the data, present and interpret results, and finalise the conclusions.	Partly - it might be helpful to introduce dataset before the filtering codes.
Is the usecase formatted like a tutorial?	It should guide a reader step by step from the scenario and learning objectives to data preparation, analysis, results, and conclusion. Each section should explain what is being done, why it is needed, and how to interpret the output before or after the relevant code. Code should be broken into logical steps, supported by clear headings, concise explanations, and relevant charts or tables. If it does not like a tutorial, please suggest how it can be improved.	Partly - the notebook is mostly code with minimal explanation. It can be improved by....
Is the notebook visually easy to follow (headings, spacing, structure)?	• Check that headings are concise and clear. - Check that sections are logically ordered and not scattered throughout the usecase, e.g. Importing libraries. • Combine related steps in one section where appropriate, e.g. handling missing data and removing duplicates. • Use reusable functions for repeated logic. • Make meaningful and consistent variable names. • Remove or shorten long print outputs. • - Remove duplicate explanation text.	Partly - headings are clear, but “Importing Libraries” and “Data Loading” are separate short sections and should be combined. There are long print() outputs that should be shortened or replaced with a summary.
Is the formatting consistent with the template? Does every piece of text have the appropriate CSS applied to it?	Compare the notebook with MOP usecase-TEMPLATE . In addition to the checklist items, verify the template-specific requirements: author and creation-date metadata, ensuring that the creation date is not incorrectly shown as the review date; unnumbered top sections in the required order; clickable and up-to-date Contents links; automatic heading numbering and correct heading hierarchy; explanatory text directly below every heading; consistent use of the Template’s CSS, typography, colours, and chart styling; and removal of all placeholders and instructional text referring to “MOP template” or “template”. Suggest what can be fixed and be specific.	Partly - CSS style numbered subsections are missing; Background and Contents are missing.
Are abbreviations & acronyms clearly defined? If not, please specify.	Define abbreviations the first time they appear. Specify where this is. Ensure that notebook abbreviations do not conflict with common meanings (e.g. KNN, SVM) or use the same abbreviation to represent different terms. After an abbreviation has been defined, use the abbreviation consistently throughout the remainder of the notebook instead of	No - define “UHI” as “Urban Heat Island” on first use in the “Scenario” section.

QA Checklist V2 headers	Explanations	Example
	switching back and forth between the full term and abbreviation.	
Is the whole usecase in Australian English (no 'z's).	Check spelling, grammar, and Australian English usage, e.g., visualization, analyse, organize, realize, optimize, summarize, centralize, maximize, utilize.	No - e.g., “visualization” in the Conclusion section.
Are there any ethical/privacy concerns noted? If so, please specify.	Check whether privacy, bias, limitations, responsible use, or sensitive data considerations are discussed where relevant, especially when the notebook uses data from sources other than City of Melbourne Open Data. Confirm that external data sources are appropriate to use, properly attributed, and do not expose personal, confidential, or sensitive information. - Privacy: Protecting personal information. For example, do not publish names, exact home addresses, phone numbers, or GPS locations that could identify an individual. - Bias: A systematic imbalance in the data or analysis results. For example, social-media data may overrepresent active users and may not represent all Melbourne residents. - Limitations: Constraints of the data or method. For example, external data may cover only certain years, have incomplete samples, contain inaccurate locations, or be unable to prove cause and effect. - Responsible use: Using data fairly, transparently, and without causing harm. For example, do not label or stigmatise a particular community, or make unfair decisions based on incomplete data. - Sensitive data: Information that could create privacy, discrimination, or harm risks. Examples include health status, race, religion, crime-victim information, children’s data, precise locations, and personal financial information.	Partly - the notebook uses external social-media location data but does not discuss privacy or responsible use. Add a limitation statement explaining that the data may not represent all Melbourne residents, may contain location bias, and must not be used to identify or target individual users. “Not required.” --If no ethical/privacy concerns.
Are there any extra improvement suggestions that you noticed that were not picked up in this checklist? If so, please specify.	Record additional useful improvements not captured by other checklist columns.	Yes - Briefly mention limitations and responsible interpretation of demographic data and how they could be improved.

3. Initial Review Process

Step 1 — Go to Checklist and start reviewing

- Open [usecase Quality Tracker](#) and go to ‘QA Checklist V2’ sheet.
- Find a usecase that has not been reviewed yet, start from the top. The older usecases should be reviewed first.
- Select your name from the “Reviewer” dropdown first to prevent two Quality Managers auditing the same notebook.
- Set “Review Status (Dropdown)” to ‘Review in Progress’ so the tracker reflects the correct status of each usecase review.

Step 2 — [Locate the usecases .ipynb file in the GitHub repository-usecases:](#)

- Locate the notebook .ipynb file according to the usecase code, for example, UC00010, using the 'Go to file' search box in the top right. Notebooks may be under status folders such as READY TO PUBLISH or UPDATE NEEDED and, in a Domain/Year/Trimester path.
- Read the notebook once from top to end. Understand the scenario and background, datasets, code flow, outputs, and conclusions before recording findings.

Step 3 — [Complete the review step by step using evidence-based findings](#)

- Review the notebook against every column in the [usecase Quality Tracker](#) QA Checklist V2 as explained above. Inspect the actual markdown, code, cell outputs, charts, data sources, and template compliance.
- Record a concise verdict in the relevant checklist column in QA Checklist V2, leave your auditing feedback. For example: 'Yes,' 'No — [reason],' 'Partly — [reason],' or 'Not required.' Do **Not** leave "Partly" / "No" only and make sure you explain if anything needs to be updated.

Step 4 — [Create an MS Planner card in Microsoft Teams Planner](#)

- Refer to the [Template Card](#) under "Templates and Project Information", follow that template for the card structure and required labels.
- Put the card into the 'Backlog' bucket. Use the usecase code as the card title (for example, 'UC00010') and apply the required 'Data Science' and 'Update Needed' labels.
- Include the GitHub link for the usecase as an attachment to the Planner card
- Add the review actions to the Planner card Checklist
- Provide a detailed, evidence-based explanation in the Planner card – **Notes**. Ensure that the Notes section begins with: "Quality Reviewer: [Your Name]"

The Planner card **Checklist** and **Notes** have different purposes. Use both so the Data Scientists can update the notebook quickly while understanding the context behind each task.

Field	What it is for	Example
Checklist	A short action list on the Planner card. Each item is one fix that can be marked complete. Begin with an action verb (Add, Fix, Remove, Rebuild, Define). Keep it specific, testable, and one task per line. Do not use vague entries such as 'Improve notebook.'	1. Add a linked Data Sets section at the beginning of the notebook. 2. Fix bug (explained below). 3. Fix template formatting inconsistencies. 4. Fix spelling errors. 5. Once complete push new .ipynb file to FINALISED folder in GitHub and set quality manager and mentor as reviewers. 6. Delete old files associated with this usecase. 7. Update the usecase Index file
Notes	The detailed explanation in the notes records why a checklist task is needed and where the issue appears. Be evidence-based: identify the cell, section, function, variable, output, or chart; state what is wrong and the required fix.	Quality Reviewer: [Your Name] 1. In the Data Sets section, add the dataset URL to access datasets; the current notebook names the dataset but gives no link. 2. Fix bug: Add na=False to str.contains('DIS').

Step 5 — Link Planner card in [usecase Quality Tracker](#)

- Copy the link of the Planner card to the “Planner Card (URL)” column as a hyperlink and display the usecase code as the link text.

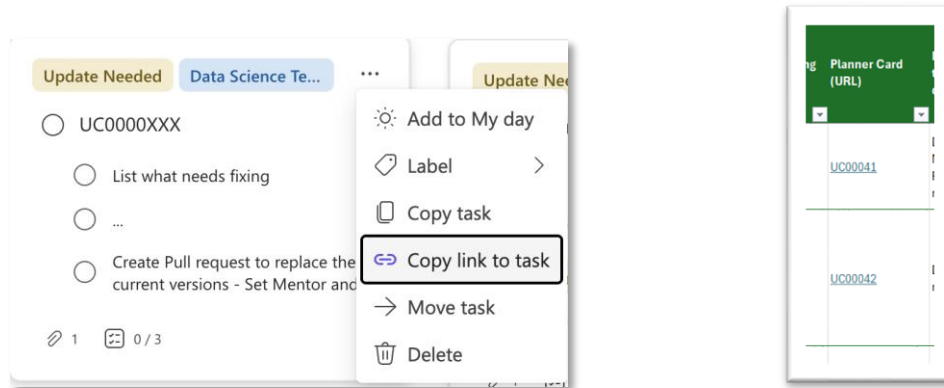


Figure 1: Copying the Planner card link (left) and pasting it into the ‘Planner Card (URL)’ column in the Usecase Quality Tracker (right).

Step 6 — Update the ‘Date Reviewed’, ‘Review Status’ and ‘Overall Rating’ in [usecase Quality Tracker](#)

- Set ‘Date Reviewed’ to the current trimester, set ‘Review Status’ to ‘Review Complete’ only after the checklist and Planner card are complete, and enter the Overall Rating out of 5.

4. Re-review and Final Approval Process

Once the initial quality review is complete and updates are required, the Data Scientist follows the process below to update the usecase, create a pull request (PR), and complete the re-review through to final approval.

Step 1 — Review the Planner card and required updates

- Open the relevant MS Planner card and review the Checklist and Notes to understand every required change.
- Set "Review Status (Dropdown)" to "Update in Progress" in the checklist
- Use the Quality Manager’s evidence and comments to identify the specific notebook sections, code, outputs, data, or formatting that need to be updated.

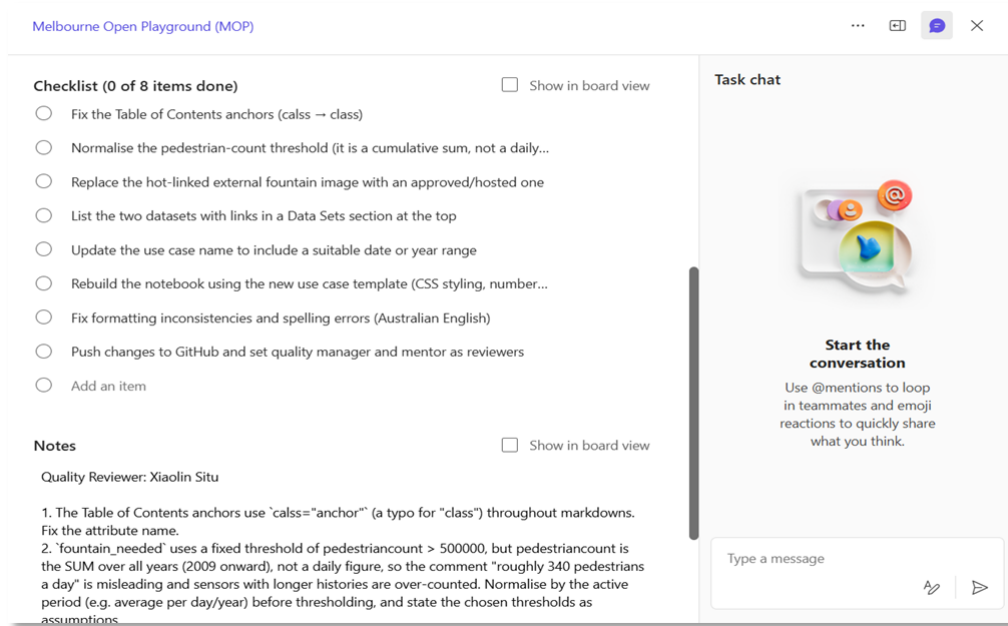


Figure 2: Microsoft Teams Planner card showing the Checklist and Notes used to identify the required updates.

Step 2 — Update the usecase and check the template

- Update the usecase using the MOP usecase-TEMPLATE and ensure the usecase code/number is included in the usecase name and relevant file names.
- Check the required structure, headings, metadata, contents, layout, formatting, and visualisations against the template.
- Review dependencies and data sources. Remove outdated or unnecessary dependencies and use the appropriate live/API data source where live data is required.

Step 3 — Complete all required fixes

- Work through every item in the Planner card Checklist and address each required update.
- Fix identified code, data, visualisation, documentation, formatting, and other issues, and make sure links and outputs work correctly.
- Ensure the notebook runs correctly from beginning to end and that the final files are complete and suitable for review.

Step 4 — Complete a self-review before creating the PR

- Check that the usecase code/name, file structure, template requirements, layout, dependencies, and data sources are correct.
- Run the notebook sequentially and check for errors, broken links, incorrect outputs, formatting issues, and unnecessary files or outputs.
- Remove or archive superseded files where required and update the usecase_Index as required.
- Confirm that every Planner card checklist item has been addressed before submitting the changes for review.

Step 5 — Create a Pull Request (PR) with a name “DS: Usecase Update: UCxxxxx”

- Push the updated .ipynb file and any required dependencies to a feature branch in the GitHub repository, delete unnecessary JSON/HTML files, update Use_Case_Index.md, rebase the branch, and create a PR for the updated use case.
- Clearly describe the changes made and explain how the Planner card checklist items have been addressed.
- Add the Quality Manager and mentor as reviewers.

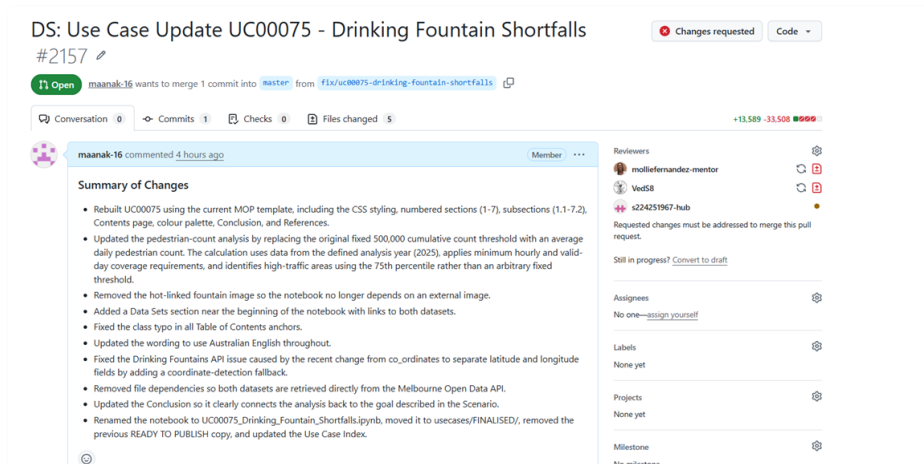


Figure 3: GitHub pull request showing the updated usecase, review status, and assigned reviewers.

Step 6 — Quality Manager re-reviews the updated usecase

- The Quality Manager reviews the PR against the original QA findings and Planner card requirements.
- If issues remain, provide specific feedback on the PR and return it to the Data Scientist for further updates. Repeat the re-review until all required issues are resolved.

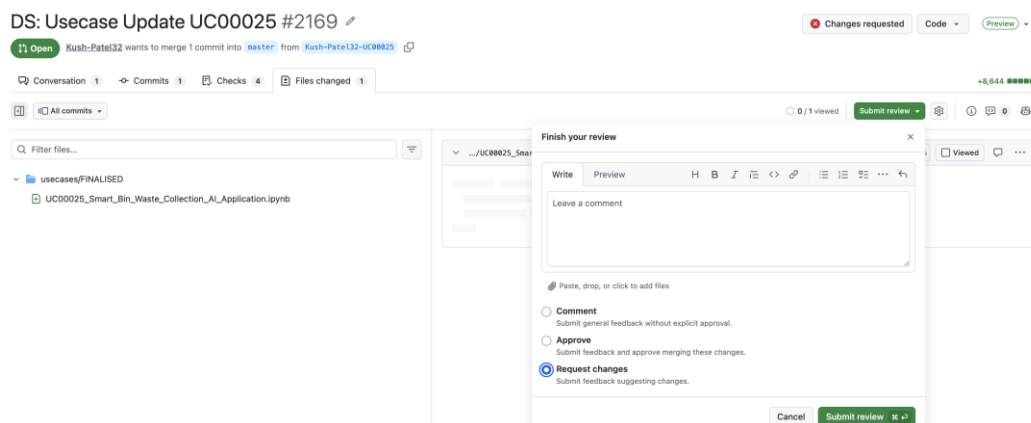


Figure 4: Submit Reviews with Request changes in Pull Request.

Step 7 — Approve and merge the PR

- Once all required changes are satisfactory, the Quality Manager and mentor approve the PR.
- Merge the approved changes into the appropriate GitHub repository/folder and confirm that the final notebook and associated files are in the correct location.

Step 8 — Complete the review

- Set 'Review Status' to 'Update Complete' in checklist once all required updates have been completed and merged.
- Confirm that the Planner card Checklist is complete and close the Planner card.

5. Common Recurring Issues

Common issues to check during review:

- Template not followed - missing CSS styling, numbered headings, required descriptions. Insufficient explanation of what happens in the usecase.
- Datasets listed in code or references instead of a linked Data Sets section at the top.

- No date/year in the usecase name.
- Hardcoded local/personal paths or API-key files.
- Data not filtered to the subject.
- Global warning suppression.
- Input() prompts that halt execution and break static HTML rendering, or raw debug prints.
- Usecase does not link back to the problem stated in the scenario throughout the analysis and conclusion of the usecase.
- Visualisations are not using the colours defined in the template.

Common issues to check during re-review:

- Use case details not updated to the current template: Missing or incorrect Author, Created, Updated, Time to Complete, Level or Pre-requisite Skills.
- Outdated CSS/template formatting: Incorrect div tags, numbering, heading hierarchy, colours or body text formatting.
- Use Case Index not updated: Use case missing from the index or positioned incorrectly.
- File organisation and naming issues: Final notebook not in the FINALISED folder, inconsistent filename/title/UC code, or old HTML, JSON or duplicate files not removed.
- Contents page issues: Incorrect sections included, links not working, unnecessary bullet points, or incorrect section numbering.
- Dataset issues: Incomplete dataset names, incorrect links, or data not retrieved using the required API method.
- Visualisation issues: Incorrect template colours, unclear headings or legends, maps not displaying correctly, or unsuitable default zoom.
- Unclean notebook outputs: Errors, warnings, deprecation messages, unnecessary outputs or empty/redundant code cells.
- Insufficient explanation: Code sections do not clearly explain what is being done and why.
- Conclusion issues: Does not follow the template, address the original scenario, summarise findings appropriately, or acknowledge relevant limitations.
- Previous feedback only partially addressed: Re-check each previous review comment individually and confirm the actual change rather than relying only on the author's summary of updates.

6. Resources

Confirm you can open all four before you start.

- Checklist Excel: [Usecase Quality Tracker](#), 'QA Checklist V2' is the review sheet.



Figure 4: Checklist Excel — Usecase Quality Tracker, 'QA Checklist V2' is the review sheet.

- GitHub repository (notebooks): [github-usecases](#)

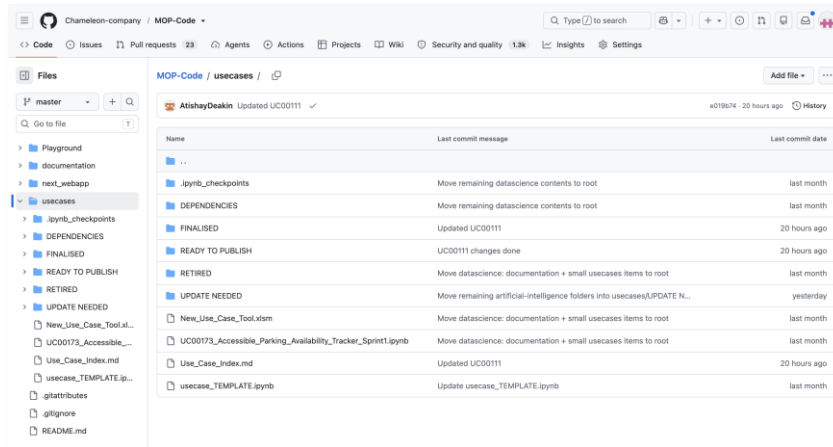


Figure 5: GitHub repository (notebooks) — *github-usecases*.

- MOP usecase template (required format): [MOP usecase-TEMPLATE](#)

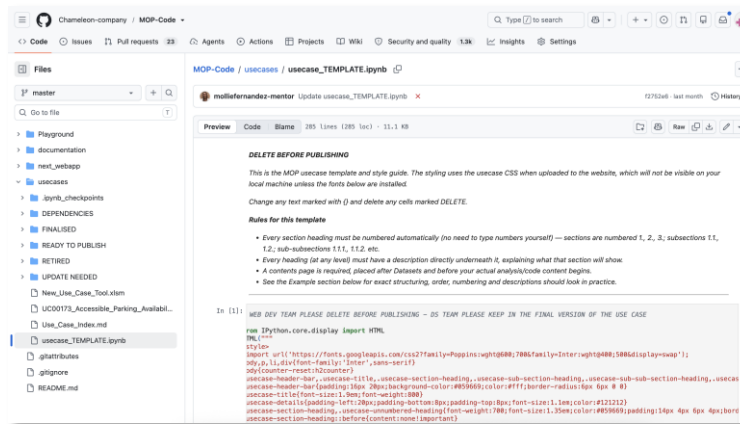


Figure 6: MOP usecase template (required format) — *MOP usecase-TEMPLATE*.

- Microsoft Teams Planner: [Melbourne Open Playground \(MOP\)](#)

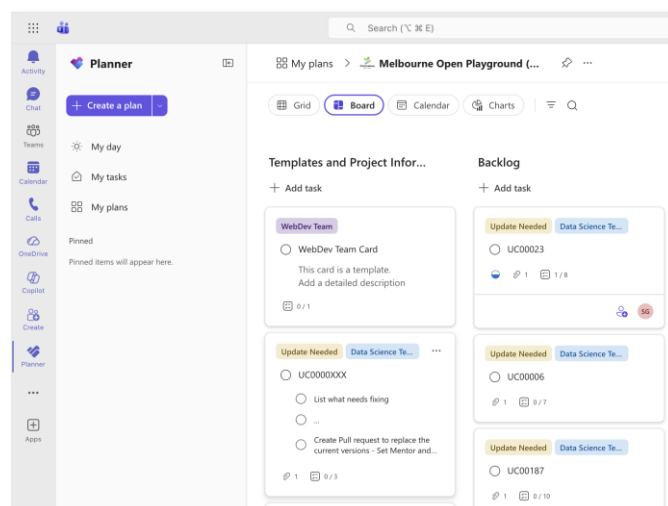


Figure 7: Microsoft Teams Planner — *Melbourne Open Playground (MOP)*.

Appendix A: Domain Selection Guide

Use this guide when selecting **Domain (Dropdown)** in QA Checklist V2. Select one domain only: choose the domain that best reflects the usecase's primary scenario, analysis, and intended outcome.

Borderline cases: Select the domain based on the "Scenario" and "Conclusion" sections. For example, classify a flood-analysis notebook as Safety and Security when its primary purpose is emergency response and risk reduction; classify it as Environmental Sustainability when its primary purpose is water systems, climate resilience, or environmental impact.

Domain	Select the domain that the usecase focuses on...	Example usecase
Business and Economy	business activity, employment, commercial activity, local economic performance, or economic decision-making.	UC00049
Community and Social Impact	community participation, inclusion, accessibility, equity, social connection, public services, or community quality of life.	UC00002
Education and Teaching	teaching resources, student learning, teacher workforce planning, educational services, school capacity, learning outcomes, academic research, or evidence to support educational planning.	UC00145
Environmental Sustainability	climate, biodiversity, waste, water, air quality, energy, urban heat, tree canopy, ecosystems, or environmental resilience.	UC00003
Health and Wellbeing	physical or mental health, health services, exercise, recreation, or public-health conditions affecting wellbeing.	UC00058
Safety and Security	crime, accidents, emergency response, hazards, public safety, disaster preparedness, or risk reduction.	UC00104
Tourism and Hospitality	visitor activity, attractions, events, accommodation, hospitality venues, or tourism planning.	UC00008
Transport and Mobility	travel, parking, walking, cycling, public transport, traffic, roads, accessibility of movement, or freight and logistics.	UC00001
Urban Planning and Development	land use, housing, built environment, infrastructure planning, development, zoning, urban growth, or neighbourhood design.	UC00010

Author: XIAOLIN SITU, MAANAK GADIA

Checker: MOLLIE FERNANDEZ, LIA ANNIE JOHN